

Proyecto final del módulo

Grupo 04 Mod. 2 Manejo y Preparación de Datos

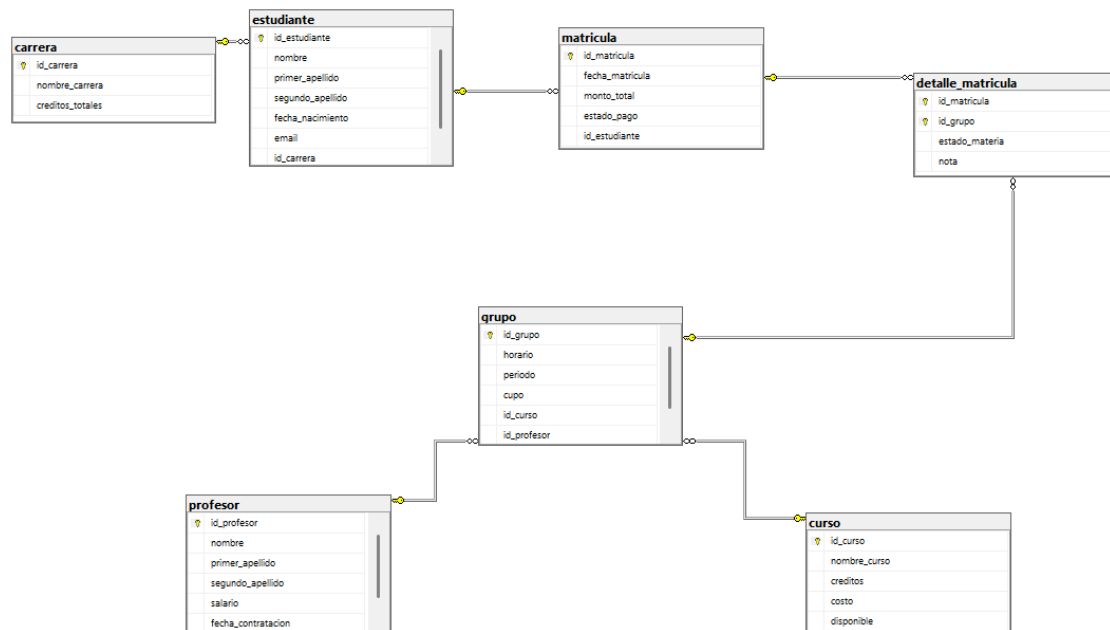
Estudiante: Keneth Josue Varela Ruíz

Profesor: Alonso Berman Moya Umaña

Incoex Instituto Tecnológico

2026

Diagrama de la base de datos



Script SQL

--Command to drop the database if it exists

--Comando que sirve para eliminar la base de datos si existe

```
DROP DATABASE IF EXISTS db_universidad;
```

```
GO
```

--Command to create the database

--Comando para crear la base de datos

```
CREATE DATABASE db_universidad;
```

```
go
```

--Command to use the database to execute queries on it

--Comando para usar la base de datos para poder ejecutar los query en esa BD

```
USE db_universidad;
```

```
go
```

----- Table Creation / Creacion de las tablas -----

-- 1. table: carrera (career)

-- 1. tabla carrera

```
create table carrera (
```

```
    id_carrera int primary key,
```

```
    nombre_carrera varchar(100) not null,
```

```

    creditos_totales int not null
);
-- 2. table: estudiante (student)
-- 2. tabla estudiante
create table estudiante (
    id_estudiante int primary key,
    nombre varchar(50) not null,
    primer_apellido varchar(50) not null,
    segundo_apellido varchar(50) not null,
    fecha_nacimiento date not null,
    email varchar(100) null,
    id_carrera int not null,
    foreign key (id_carrera) references carrera(id_carrera)
);

-- 3. table: profesor (professor)
-- 3. tabla profesor
create table profesor (
    id_profesor int primary key,
    nombre varchar(50) not null,
    primer_apellido varchar(50) not null,
    segundo_apellido varchar(50) not null,
    salario decimal(10,2) not null,
    fecha_contratacion date not null
);

-- 4. table: curso (course)
-- 4. tabla curso
create table curso (
    id_curso int primary key,
    nombre_curso varchar(100) not null,
    creditos int not null,
    costo decimal(10,2) not null,
    disponible bit not null
);

-- 5. table: grupo (course groups)
-- 5. tabla grupo
create table grupo (
    id_grupo int primary key,
    horario varchar(50) not null,
    periodo varchar(20) not null,
    cupo int not null,
    id_curso int not null,
    id_profesor int not null,
    foreign key (id_curso) references curso(id_curso),
    foreign key (id_profesor) references profesor(id_profesor)
);

```

```

-- 6. table: matricula (enrollment)
-- 6. tabla matricula
create table matricula (
    id_matricula int primary key,
    fecha_matricula date not null,
    monto_total decimal(10,2) not null,
    estado_pago varchar(20) not null,
    id_estudiante int not null,
    foreign key (id_estudiante) references estudiante(id_estudiante)
);
-- 7. table: detalle_matricula (enrollment details)
-- 7. tabla detalle_matricula
create table detalle_matricula (
    id_matricula int not null,
    id_grupo int not null,
    estado_materia varchar(20) not null,
    nota decimal(4,2) null,
    primary key (id_matricula, id_grupo),
    foreign key (id_matricula) references matricula(id_matricula),
    foreign key (id_grupo) references grupo(id_grupo)
);

```

----- Data Insertion / Inserts de los datos -----

INSERT INTO carrera VALUES

```

(1,'Informática Empresarial',140),
(2,'Administración de Empresas',130),
(3,'Contabilidad',135),
(4,'Ingeniería Industrial',150),
(5,'Derecho',145);

```

INSERT INTO estudiante VALUES

```

(1,'Juan','Pérez','Rojas','2002-05-14','juan@gmail.com',1),
(2,'María','Gómez','Soto','2001-08-20','maria@gmail.com',2),
(3,'Carlos','Ramírez','Vargas','2003-03-10',NULL,1),
(4,'Ana','López','Mora','2000-11-18','ana@gmail.com',3),
(5,'Luis','Castro','Jiménez','2002-07-25',NULL,4),
(6,'Sofía','Hernández','Rojas','2001-09-09','sofia@gmail.com',5),
(7,'Diego','Araya','Salas','2003-01-15','diego@gmail.com',2),
(8,'Valeria','Mora','Chaves','2002-12-30','valeria@gmail.com',1);

```

INSERT INTO profesor VALUES

```

(1,'Pedro','Sánchez','Vega',1200000,'2018-02-10'),
(2,'Laura','Jiménez','Castro',1450000,'2016-08-15'),
(3,'Mario','Rojas','Solano',980000,'2020-01-20'),
(4,'Patricia','Alvarado','Mora',1600000,'2015-05-05'),
(5,'Andrés','Quesada','León',1100000,'2019-09-12');

```

```

INSERT INTO curso VALUES
(1,'Bases de Datos',4,95000,1),
(2,'Programación I',4,90000,1),
(3,'Redes',3,85000,0),
(4,'Contabilidad General',3,80000,1),
(5,'Derecho Empresarial',4,92000,1),
(6,'Investigación de Operaciones',4,97000,0),
(7,'Desarrollo Web',4,98000,1),
(8,'Matemática',3,70000,1);

```

```

INSERT INTO grupo VALUES
(1,'Lunes 8:00-10:00','I-2026',30,1,1),
(2,'Martes 10:00-12:00','I-2026',25,2,2),
(3,'Miércoles 1:00-3:00','I-2026',20,3,3),
(4,'Jueves 8:00-10:00','I-2026',30,4,4),
(5,'Viernes 2:00-4:00','I-2026',35,5,5),
(6,'Lunes 10:00-12:00','II-2026',30,6,4),
(7,'Martes 1:00-3:00','II-2026',25,7,2),
(8,'Viernes 8:00-10:00','II-2026',40,8,1);

```

```

INSERT INTO matricula VALUES
(1,'2026-01-10',185000,'Pagado',1),
(2,'2026-01-11',175000,'Pendiente',2),
(3,'2026-01-12',95000,'Pagado',3),
(4,'2026-01-12',160000,'Pagado',4),
(5,'2026-01-13',187000,'Pendiente',5),
(6,'2026-01-13',98000,'Pagado',6),
(7,'2026-01-14',180000,'Pagado',7),
(8,'2026-01-14',165000,'Pendiente',8);

```

```

INSERT INTO detalle_matricula VALUES
(1,1,'Aprobado',92),
(1,2,'Aprobado',88),
(2,2,'Cursando',NULL),
(2,3,'Cursando',NULL),
(3,1,'Reprobado',58),
(4,4,'Aprobado',95),
(4,5,'Aprobado',90),
(5,6,'Cursando',NULL),
(5,7,'Cursando',NULL),
(6,8,'Aprobado',87),
(7,1,'Aprobado',91),
(7,7,'Aprobado',94),
(8,2,'Reprobado',55),
(8,8,'Aprobado',80);

```

----- SQL Queries / Consultas SQL -----

-- Select statements for all tables

--Selects de las tablas

Select * from carrera

select * from curso

select * from estudiante

select * from profesor

select * from grupo

select * from matricula

select * from detalle_matricula

--SELECT: First name, last name, email, and date of birth of students

--SELECT: Nombre, apellido, email y la fecha de nacimientos de los estudiantes

Select e.nombre, e.primer_apellido, e.email ,e.fecha_nacimiento
from estudiante e

--WHERE: Name and cost of courses where the cost is greater than 90000

--WHERE: Nombre y costo de los cursos donde su costo es mayor a 90000

select c.nombre_curso, c.costo
from curso c
where costo > 90000;

--ORDER BY: First name, last name, and salary of professors ordered in descending order
by salary

--ORDER BY: Nombre, primer apellido y salario de los profes ordenados de forma
descendiente segun su salario

select p.nombre, p.primer_apellido, p.salario
from profesor p
order by salario desc;

--DISTINCT: Display distinct payment statuses for enrollments

--DISTINC: Mostrar los estados estados distintos del estado de pago de matricula

select distinct m.estado_pago
from matricula m;

--TOP: Name and cost of the top 3 most expensive courses ordered in descending order

--TOP: nombre y costo de los 3 cursos mas caros ordenados de forma descendiente

select top 3 c.nombre_curso, c.costo
from curso c
order by costo desc;

--LIKE: Full name of students whose first last name starts with R

--LIKE:nombre completo de los estudiantes donde el primer apellido empieza con R)

select e.nombre, e.primer_apellido, e.segundo_apellido
from estudiante e
where primer_apellido like 'r%';

--BETWEEN: Data of professors with salary between 1000000 and 1500000

--BETWEEN: datos de los profes con salario entre 1000000 y 1500000

```
select * from profesor
where salario between 1000000 and 1500000;
```

--IN: First name, first last name, and major id of students enrolled in majors with id (1, 2, and 5)

--IN: nombre, primer apellido y el id de carrera de los estudiantes de las carreras con id(1, 2 y 5)

```
select e.nombre, e.primer_apellido, e.id_carrera
from estudiante e
where id_carrera in (1, 2, 5);
```

--NOT: Available courses

--NOT: Cursos que estan disponibles

```
select c.nombre_curso, c.disponible
from curso c
where not disponible = 0;
```

--IS NULL: Full name and email of students who do not have an email

--IS NULL: nombre completo y el email de los estudiantes que no tienen email

```
select e.nombre, e.primer_apellido, e.segundo_apellido, e.email
from estudiante e
where email is null;
```

--IS NOT NULL: Enrollment id, group id, and grade for completed courses

--IS NOT NULL: id de matricula, id de grupo, y nota de los cursos ya finalizados

```
select dm.id_matricula, dm.id_grupo, dm.nota
from detalle_matricula dm
where nota is not null;
```

--AND: First name, first last name, major id, and date of birth of students in major 1 born after June 2002

--AND: nombre, primer apellido, id de la carrera y fecha de nacimientos de los estudiantes de la carrera con id 1 y que nacieron despues de junio del 2002

```
select e.nombre, e.primer_apellido, e.id_carrera, e.fecha_nacimiento
from estudiante e
where id_carrera = 1 and fecha_nacimiento > '2002-06-01';
```

--OR: id, total amount, and payment status of enrollments that are pending or total amount is greater than or equal to 180000

--OR: id, monto total y estado de pago de las matriculas que estan pendientes o el monto total es mayor o igual a 180000

```
select m.id_matricula, m.monto_total, m.estado_pago
from matricula m
where estado_pago = 'Pendiente' or monto_total >= 180000;
```

--GROUP BY: Total number of students per career

--GROUP BY: Total de estudiantes de cada carrera
select e.id_carrera, count(*) as total_estudiantes
from estudiante e
group by id_carrera;

--HAVING: Total number of students for careers with 2 or more students
--HAVING: total de estudiantes de las carreras que tienen 2 o mas estudiantes
select e.id_carrera, count(*) as total_estudiantes
from estudiante e
group by id_carrera
having count(*) >= 2;

--COUNT, SUM, AVG, MIN, MAX: Course count and aggregations on course costs
--COUNT, SUM, AVG, MIN, MAX: Cantidad de cursos y operaciones con el costo de los cursos
select count(*) as total_cursos,
sum(c.costo) as costo_total_catalogo,
avg(c.costo) as costo_promedio,
min(c.costo) as curso_mas_barato,
max(c.costo) as curso_mas_caro
from curso c;

--INNER JOIN: First name and first last name of students and their associated career
--INNER JOIN: nombre y primer apellido de los estudiantes y a que carrera pertenecen
select e.nombre, e.primer_apellido, c.nombre_carrera
from estudiante e
inner join carrera c on e.id_carrera = c.id_carrera;

--LEFT JOIN: Display all careers and the students belonging to them
--LEFT JOIN: Mostrar todas las carreras y los estudiantes que pertenecen a ellas
select c.nombre_carrera, e.nombre, e.primer_apellido
from carrera c
left join estudiante e on c.id_carrera = e.id_carrera;

--RIGHT JOIN: Display all groups and their assigned professor
--RIGHT JOIN: Mostrar todos los grupos y el profesor asignado
select g.id_grupo, g.horario, p.nombre, p.primer_apellido
from profesor p
right join grupo g on p.id_profesor = g.id_profesor;

--Subqueries: Students enrolled in the major with the highest total credits
--Subconsultas: Estudiantes matriculados en la carrera con más créditos totales
select e.nombre, e.primer_apellido, e.id_carrera
from estudiante e
where id_carrera = (select top 1 c.id_carrera
from carrera c
order by credits_totales desc
);

----- Views / Vistas -----

--View to display academic records of students

--Vista para ver el expediente academico de los estudiantes

create view expediente_academico

as

select e.nombre + ' ' + e.primer_apellido + ' ' + e.segundo_apellido as estudiante,

c.nombre_curso,g.horario,

dm.estado_materia,dm.nota

from estudiante e

inner join matricula m on e.id_estudiante = m.id_estudiante

inner join detalle_matricula dm on m.id_matricula = dm.id_matricula

inner join grupo g on dm.id_grupo = g.id_grupo

inner join curso c on g.id_curso = c.id_curso;

select * from expediente_academico

--View to display course information (name, assigned professor, period, schedule, and capacity)

--Vista para ver la informacion de los cursos (nombre, profesor que lo imparte, periodo, horario y cupos)

create view informacion_cursos

as

select g.id_grupo,c.nombre_curso,p.nombre + ' ' + p.primer_apellido + ' ' +

p.segundo_apellido as profesor, g.horario,g.periodo,g.cupo

from grupo g

inner join curso c on g.id_curso = c.id_curso

inner join profesor p on g.id_profesor = p.id_profesor;

select * from informacion_cursos

--View to display total assigned groups and schedule per professor

--Vista para ver el total de grupos a los que un profesor esta asignado y su horario

create view carga_profesores

as

select p.id_profesor,p.nombre + ' ' + p.primer_apellido as profesor,count(g.id_grupo) as total_grupos,

string_agg(c.nombre_curso + ' (' + g.horario + ')', ', ') as horario

from profesor p

left join grupo g on p.id_profesor = g.id_profesor

left join curso c on g.id_curso = c.id_curso

group by p.id_profesor, p.nombre, p.primer_apellido;

select * from carga_profesores

--View to display student payment report

--Vista para ver el reporte de pagos de los estudiantes

```
create view reporte_pagos
as
select m.id_matricula,e.nombre + ' ' + e.primer_apellido + ' ' + e.segundo_apellido as
estudiante,
m.fecha_matricula,m.monto_total,m.estado_pago
from matricula m
inner join estudiante e on m.id_estudiante = e.id_estudiante;

select * from reporte_pagos
```

Enlace al Github: https://github.com/JosueVR17/Ciencia_de_Datos_Keneth_Varela